ROOTCAUSE SERVER

17 · Resumen ejecutivo

Documento: 17-executive-summary.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO

- Qué es
- Qué necesidad cubre
- Quién lo usa
- Capacidades principales
- Tres decisiones que definen el producto
- Tecnologías
- Arquitectura, en tres frases
- Estado actual
- Fortalezas
- Riesgos
- Oportunidades de mejora
- Próximos pasos recomendados
  - Antes de la próxima versión
  - En la versión siguiente
  - Cuando se toque la zona
- Conclusión

> Presentación del sistema para dirección, clientes, evaluadores y posibles colaboradores. Sin detalle técnico innecesario: solo lo que sirve para decidir.
>
> Base: análisis completo del repositorio el 2026-08-26, versión `0.2.0`, commit `2efedb2`.

Qué es

RootCause Server es una herramienta que vigila un conjunto de servidores y la red que los rodea, y explica cada problema que encuentra con la evidencia que lo demuestra.

Consta de dos programas: un **sensor** que se instala en cada servidor y solo observa, y un **plano de control** que reúne lo observado, lo interpreta con 18 reglas y lo presenta en una consola web.

Es el miembro de servidor de la familia RootCause, junto a herramientas equivalentes para estaciones Windows y macOS, navegador, móvil, códigos QR, custodia Bitcoin y contratos blockchain.

Qué necesidad cubre

Un servidor ya escribe registros. La aplicación tiene el suyo, la base de datos el suyo y el sistema operativo el suyo. Cada uno ve su parte, **y ninguno ve la frase completa**:

> Un puerto de base de datos publicado en Internet. Una ráfaga de intentos de acceso desde una sola dirección. Veinte minutos después, una sesión concedida a esa misma dirección.

Ninguno de los tres hechos es un incidente por separado. Los tres juntos, sí. Nadie los está leyendo juntos.

RootCause Server no reimplementa lo que ya existe: **correlaciona lo que nadie está mirando junto y lo dice en una frase accionable.**

Quién lo usa

Perfil	Para qué
Administradores de sistemas y SRE	Ver qué está expuesto y decidir qué hacer
Responsables de seguridad	Superficie de la flota, presión sobre la autenticación, postura
Auditores	Descargar evidencia y revisar el registro de cambios
Equipos de plataforma	Integrar métricas y sondas de salud en el monitoreo existente

Encaja especialmente bien en organizaciones **pequeñas y medianas** que administran entre unos pocos y unas decenas de servidores, sin presupuesto ni equipo para operar un SIEM.

Capacidades principales

Capacidad	Qué responde
Superficie expuesta	Qué servicio de qué equipo es alcanzable desde fuera, y hasta dónde llega
Presión sobre la autenticación	Quién está empujando contra la puerta, y **si consiguió entrar**
Integridad de archivos críticos	Si cambió algo que decide quién entra al servidor
Controles básicos	Firewall activo, parches pendientes, reloj sincronizado
Disponibilidad del propio sensor	Si un agente dejó de reportar —lo que se investiga como manipulación hasta demostrar lo contrario—
Saturación con memoria	CPU sostenida, presión de memoria, disco y **cuántas horas faltan para llenarlo**
Respuesta guiada	El comando exacto que arregla el problema, y cómo deshacerlo
Evidencia exportable	Un paquete con activos, incidentes y auditoría

Tres decisiones que definen el producto

1 · El alcance multiplica el riesgo. La misma base de datos accesible solo desde el propio servidor es rutina; accesible desde Internet es una emergencia. El producto distingue el alcance, no solo el hecho, y ajusta la gravedad según el papel declarado del equipo.

2 · Nunca reporta cero donde no pudo mirar. Si el sensor no logra revisar el firewall, la consola dice «no se pudo inspeccionar el firewall», no «firewall correcto». Toda puntuación viaja con la lista de lo que no se revisó. Es un requisito formal del proyecto, con pruebas automáticas que lo hacen cumplir.

3 · Diagnostica; no interviene. Escribe el runbook con los comandos exactos y su forma de revertirlos, e **indica quién debe ejecutarlo: una persona**. No bloquea direcciones por su cuenta, no edita configuraciones y no reemplaza al antivirus, al EDR ni al firewall. Una prueba automática hace fallar la compilación si alguien introduce un comando destructivo.

Tecnologías

Todo está escrito en **Rust**, un lenguaje elegido por seguridad de memoria y por producir un binario único que funciona igual en Windows, Linux y macOS. El proyecto **prohíbe** el uso de código sin comprobaciones de seguridad (`unsafe`), y trata cada advertencia del analizador como un defecto.

Los datos se guardan en **SQLite**, una base de datos que es un solo archivo: no hay servidor de base de datos que instalar, configurar ni endurecer. La consola web va **compilada dentro del programa**, así que no depende de ningún servicio externo ni carga nada de Internet.

Arquitectura, en tres frases

Un sensor por servidor envía lo que observa al plano de control. El plano de control lo valida, lo guarda tal cual llegó y le aplica un motor de reglas que **no toca el mundo exterior**: no abre conexiones, no lee archivos y no consulta el reloj. Con la misma información y las mismas reglas produce siempre el mismo resultado, y por eso un problema detectado hoy se puede reexaminar dentro de seis meses contra la evidencia guardada.

Estado actual

Indicador	Valor
Versión	`0.2.0`
Madurez	Base funcional sólida, no plataforma empresarial terminada
Reglas de detección	18, publicadas por el propio programa
Pruebas automatizadas	**250, todas en verde** el 2026-08-26
Plataformas verificadas	Windows, Linux y macOS, en cada cambio
Controles automáticos de calidad	15 en integración continua, más 5 de seguridad
Documentación previa	15 documentos, 3 decisiones de arquitectura, 3 requisitos de seguridad
Licencia	MIT
Telemetría enviada a terceros	**Cero**

Fortalezas

1 · Honestidad verificable. El producto publica sus límites en lugar de disimularlos, y los números que afirma la documentación los comprueba un script en cada compilación. «18 reglas» es un hecho verificado, no un recuerdo.

2 · Disciplina de ingeniería fuera de lo común para su tamaño. Sin código inseguro. Analizador sin advertencias en tres sistemas operativos. Pruebas contra la versión mínima **y** la estable de Rust. Auditoría de dependencias, inventario de componentes, escaneo de secretos en el histórico y análisis de la imagen de contenedor. Y una puerta final que hace fallar la rama si **cualquier** control quedó sin ejecutarse —no solo si falló—.

3 · Seguro por omisión. Escucha solo en el propio equipo. Sin credencial no arranca. Se defiende a sí mismo con límite de peticiones y bloqueo, evaluados **antes** de comparar la credencial. Toda opción que debilita la protección hay que escribirla a mano, y las combinaciones peligrosas se rechazan.

4 · Cero dependencia de terceros en funcionamiento. No envía nada fuera del despliegue del cliente, no carga recursos externos y no requiere ninguna cuenta ni suscripción.

5 · Evidencia recuperable. Los datos originales se conservan tal como llegaron, así que una regla nueva puede reexaminar un incidente antiguo sin pedirle nada a los servidores.

Riesgos

Del análisis completo (documento 15), los que importan para decidir:

Riesgo	Consecuencia	Gravedad
Un tope interno de 500 hallazgos afecta a la puntuación, las métricas y el paquete de evidencia sin avisarlo	Un informe podría calcularse sobre menos datos de los que hay	Alta
Sin respaldo automatizado	La evidencia acumulada no se puede regenerar si se pierde	Alta en consecuencia, baja en probabilidad
Una sola credencial, sin rotación	Cambiarla obliga a reconfigurar todo a la vez, con interrupción	Alta
Identidad del servidor basada en su nombre	Dos servidores con el mismo nombre se mezclan	Alta en entornos con autoescalado
Sin usuarios ni permisos	Quien tenga la credencial ve todo	Media, y es una decisión declarada
Una sola instancia	Cada actualización deja un hueco de unos segundos en la vigilancia	Baja

Ninguno es un fallo de seguridad explotable. Son límites de una versión `0.2`, y la mayoría están reconocidos por el propio proyecto.

Oportunidades de mejora

Oportunidad	Beneficio	Esfuerzo
Respaldo automatizado y prueba de restauración	Protege lo único irreemplazable del sistema	**Bajo**
Corregir el tope de 500 en informes y exportación	Elimina el riesgo más relevante encontrado	Bajo–medio
Pruebas de robustez para el código que lee la salida de herramientas del sistema	Cubre la única zona con incertidumbre técnica real	Medio
Rotación de credenciales con solapamiento	Permite cambiarla sin interrupción	Medio
Notificaciones salientes (correo, webhook)	Hoy hay que consultar la consola o montar Prometheus	Medio
Pruebas de la consola web	1 177 líneas sin cobertura	Medio–alto
Identidad de servidor configurable	Habilita entornos con autoescalado	Bajo

Próximos pasos recomendados

Antes de la próxima versión

1. Añadir respaldo automatizado y **verificar que restaura**.
2. Corregir el tope de 500 hallazgos en la exportación de evidencia y en el cálculo de la puntuación.

En la versión siguiente

1. Pruebas de robustez para los lectores de salida de herramientas del sistema.
2. Rotación de credenciales sin interrupción.
3. Identidad de servidor configurable, para entornos con nombres repetidos.
4. Hacer atómico el ciclo completo de ingesta.

Cuando se toque la zona

1. Pruebas de la consola web.
2. Notificaciones salientes.
3. Limpieza menor: dependencias declaradas y no usadas, directorio vacío, nombre desactualizado de un paso de la automatización.

Conclusión

RootCause Server es una **base funcional sólida y honesta**, con una disciplina de verificación superior a la habitual en proyectos de su tamaño y con sus límites publicados en lugar de disimulados. Las 250 pruebas pasan, los tres guardianes propios del repositorio pasan, y no se encontró código muerto, secretos filtrados ni vulnerabilidades explotables.

No es una plataforma empresarial terminada, y el proyecto no pretende que lo sea: no tiene usuarios ni permisos, no tiene alta disponibilidad y no reemplaza a un SIEM.

Para una organización que administra entre unos pocos y unas decenas de servidores y hoy no tiene ninguna visibilidad correlacionada, **el valor es inmediato y el coste de operación es bajo**: un binario, un archivo y un proxy delante.

Los dos trabajos previos recomendados —respaldo automatizado y corrección del tope de 500— son de esfuerzo bajo y eliminan los dos riesgos de mayor consecuencia.

Anterior: 16 · Glosario · **Siguiente:** 18 · Guía para un nuevo desarrollador · **Índice:** README
